



P05

UNIVERSITY DIGITAL LIBRARY MANAGEMENT SYSTEM

CIA-3 PROJECT DEVELOPMENT

Project Code: P05 | Semester: 5th Semester
Course: Advanced JavaScript Backend Frameworks
(Node.js & Express JS)

TEAM DETAILS

S. No.	Student Name	Roll No.	Department	Section
1	S Mehanath	2460645	ADSE	5BTCSAIML C
2	SHAMITH M GOWDA	2462530	ADSE	5BTCSAILML C
3	TENZIN RIGZIN	2460462	ADSE	5BTCSAIML C

GitHub Repository: <https://github.com/Mehanath20/6UNIVERSITY-DIGITAL-LIBRARY-MANAGEMENT-SYSTEM>

2. PROJECT INFORMATION & EXECUTIVE SUMMARY

PROJECT INFORMATION

Project Code: P05

Project Title: University Digital Library Management System

Domain: Education

Course: Advanced JavaScript Backend Frameworks (Node.js & Express JS)

Academic Assessment: Continuous Internal Assessment – 3 (CIA-3)

Repository: [Insert public/shared GitHub repository URL]

EXECUTIVE SUMMARY

The University Digital Library Management System (DLMS) is a role-based web application designed to digitize academic library operations. It centralizes catalog discovery, circulation, reservations, fine management, inventory accountability, membership policies, notification records and institutional reporting. The objective is not simply to provide a digital list of books, but to represent the actual operational lifecycle of a university library in which different users have different permissions and each circulation action changes persistent application state.

IMPLEMENTATION OVERVIEW

The project uses a Node.js and Express.js backend, MongoDB accessed through Mongoose, and a responsive HTML5, Bootstrap 5, CSS3 and vanilla JavaScript frontend. Authentication is handled using JWT and password hashing uses bcryptjs. Joi validation is applied before business logic, while role-based middleware restricts protected operations. The design also includes centralized error handling, security middleware, Postman testing and backend/integration testing support.

WHY THE SYSTEM IS PRACTICAL

A strong aspect of DLMS is its business-rule-driven workflow. A member cannot simply create a transaction whenever a button is clicked. The backend checks membership limits and copy availability, calculates the due date, updates the transaction and adjusts availability. Returns can generate overdue fines and affect reservation queues. Inventory changes can be audited. This makes the system suitable for demonstration as a real management application rather than a collection of static CRUD screens.

The system is intended to serve as a single operational layer for a university library. A member begins with discovery, but the same application continues into authentication, borrowing, reservations and fine visibility. Staff members use the same underlying records from a different perspective. Because the workflows share a database, the application can keep availability and transaction information synchronized rather than depending on separate manual registers.

From an evaluation perspective, the project can be demonstrated in two complementary ways. The first is the visible user journey through the browser. The second is the technical journey behind each action: authenticated HTTP request, middleware checks, controller/service logic, Mongoose operation and JSON response. Presenting both views makes it possible to explain not only what the application looks like, but why the workflow behaves correctly.

The application is structured so that the public experience and the authenticated experience form one continuous product. The landing page provides context and discovery, while login establishes a trusted identity. After authentication, the member sees personal information and library actions, whereas staff see operational controls. This separation improves usability because the interface is not overloaded with functions that do not belong to the current user. At the same time, the shared backend ensures that every role works with the same underlying catalogue and circulation state.

The report also treats implementation evidence as part of the technical deliverable. A screenshot shows the visible result, but the actual project is supported by APIs, models, validation and business rules. During a demonstration, the evaluator can therefore move from a screen to the corresponding request and database effect. This provides a stronger explanation of functionality and makes it easier to distinguish an implemented workflow from a static prototype.

3. PROBLEM STATEMENT & EXISTING CHALLENGES

PROBLEM STATEMENT

Academic libraries manage large numbers of books, members and circulation events. When these activities are handled using paper registers or disconnected spreadsheets, staff may spend unnecessary time checking availability, calculating overdue fines, maintaining reservation lists and reconciling stock. The lack of a common system also makes it difficult to determine who performed a sensitive administrative action or why a copy count changed.

MANUAL CIRCULATION CHALLENGES

Paper-based circulation records make member history slow to search and difficult to audit. A book may appear available in one register while another record shows it as issued, lost or damaged. Manual fine calculations can produce inconsistent results when overdue days or member-specific rates are interpreted differently. The absence of an explicit reservation queue can also create unfairness when several members are waiting for the same title.

INFORMATION VISIBILITY

Students need a simple way to discover books, check availability, see current loans, review fines and reserve unavailable titles. Librarians need a different view that emphasizes issue/return operations, inventory and holds. Administrators need broader controls and reports. A single undifferentiated interface would expose unnecessary complexity and could weaken access control.

DLMS RESPONSE

DLMS addresses these problems through centralized MongoDB data, role-aware interfaces, backend authorization, automated due-date and fine rules, FIFO holds, inventory audit records and operational reports. The system creates a consistent chain from user action to API request, business validation, database update and visible result.

A digital library system must solve more than the problem of displaying book titles. It must maintain state over time. The same book can be available today, issued tomorrow, returned later, and temporarily reserved by another member. A useful system therefore needs rules that connect the current state with the previous actions that produced it. DLMS addresses this by keeping transaction, inventory and reservation information as persistent records.

Another important requirement is controlled responsibility. Members should not be able to perform staff operations simply by discovering an endpoint, and staff actions should be explainable when inventory or financial information changes. The use of backend role checks, validation and audit-oriented records gives the system a stronger operational foundation than a frontend-only application.

The problem becomes more significant when a library has repeated circulation activity. A single title may have multiple physical copies, different members may request the same title, and each loan can have a different due date. If these details are not represented explicitly, staff have to reconstruct the state manually. DLMS represents the important states directly through books, transactions, holds and inventory records, allowing the system to answer common questions such as who has a book, whether a copy is available, who is waiting and whether a fine is outstanding.

The system also reduces dependency on informal staff knowledge. Instead of expecting a librarian to remember a fine policy or reservation order, the backend can retrieve the applicable policy and queue state from persistent data. This creates a more predictable operational process and makes the application easier to demonstrate, maintain and extend.

4. PROJECT OBJECTIVES & SCOPE

PRIMARY OBJECTIVES

1. Model books, members and transactions in MongoDB using sensible references.
2. Implement borrowing and return workflows with due-date and fine logic.
3. Support catalog search by title, author, ISBN and category.
4. Provide librarian-facing inventory and reporting functionality.
5. Enforce role-based access for Member, Librarian and Admin users.

ADDITIONAL OBJECTIVES

6. Maintain copy-level availability and inventory audit information.
7. Support reservations using a FIFO hold queue.
8. Track fine payments and authorized staff waivers.
9. Provide overdue, availability and hold-related notification records.
10. Deliver a GitHub-hosted codebase and Postman collection for evaluation.

PROJECT SCOPE

The project covers the complete academic-library workflow from public discovery and authentication to catalog search, book details, borrowing, returns, reservations, fines and history. Administrative scope includes catalog/inventory supervision, membership plans, staff operations and reporting. The system is therefore broad enough to demonstrate frontend development, REST APIs, database modelling, security and business rules within one coherent application.

EVALUATION ALIGNMENT

The project provides visible UI evidence as well as backend-oriented evidence. The evaluator can follow the user journey through the landing page, login, student/member dashboard, catalog components and administration screens. API and database sections explain what happens behind those screens, while Postman and negative test cases demonstrate that important rules are enforced server-side.

The objectives are deliberately connected. Catalog search supports discovery; membership plans support eligibility; circulation uses both availability and policy; returns influence fines and inventory; reservations depend on availability; and reporting depends on the records produced by all of these operations. This interconnected scope ensures that the project can be demonstrated as one complete workflow.

The scope also provides a clear boundary for future development. The current project establishes the core data model and business processes. Additional integrations such as real payment services, physical scanning devices, external messaging and multi-branch operations can be added later without changing the fundamental concept of the system.

The objectives are measurable through visible workflows and API behavior. Catalog search can be demonstrated by querying different fields, circulation can be demonstrated by changing a book from available to issued and back, and reservations can be demonstrated by creating a queue for an unavailable title. Security objectives can be tested by repeating the same operation with different roles. This provides a direct relationship between the written objectives and the evidence presented in the implementation section.

The project scope intentionally includes both user-facing and administrative capabilities. A university library needs members to interact with resources, but it also needs staff to control the shared data that makes those interactions possible. Including both sides of the workflow makes the project more complete and gives the evaluator multiple technical areas to assess, including database design, authorization, validation and reporting.

5. ACTORS / USER ROLES

MEMBER / STUDENT

Members search the catalog, open book details, borrow within their plan limits, place holds for unavailable titles, view current loans and borrowing history, review fines and receive relevant notification records. The student interface is intentionally focused on personal academic-library activity rather than administrative controls.

LIBRARIAN

Librarians operate the circulation side of the system. They can process issues and returns, manage books and inventory, handle reservation queues, review fines and access operational reports. Their interface provides more information than the member portal because their work affects shared library records.

ADMINISTRATOR

Administrators have the broadest institutional access. They can manage staff-oriented settings, membership plans, inventory/circulation oversight and reports. Administrative functions are protected through backend role checks so that direct API calls cannot bypass the frontend.

ROLE SEPARATION

JWT identifies the authenticated user, while authorization middleware checks the role before protected operations are executed. Student and faculty are represented as member categories with configurable borrowing limits, loan periods and fine policies. This design separates identity, role and membership policy instead of hard-coding every rule into the UI.

Role separation is visible in the application itself. The member interface concentrates on personal information and discovery, while staff interfaces expose operational tables and administrative actions. This is useful for usability because each user sees information that is relevant to their responsibilities. At the backend, the same separation is enforced independently through authorization middleware.

Student and Faculty are treated as membership categories rather than unrelated application roles. This distinction is useful because two members can share the same portal while following different borrowing policies. A policy change can therefore be represented through membership-plan data rather than by creating separate application code paths for every category.

A role is more than a different menu. It represents a different responsibility in the system. A member primarily consumes library resources, a librarian manages daily circulation and inventory, and an administrator manages institutional settings and broader oversight. Each role therefore requires different API permissions and different information density. The frontend communicates this distinction visually, while the backend enforces it technically.

The student and faculty categories are especially useful for demonstrating policy-driven design. Both can use the same member portal, but their membership plans can contain different borrowing limits and fine rules. This avoids creating duplicate screens for every category and shows how a single reusable workflow can adapt to policy data.

6. FUNCTIONAL MODULES – PART I

1. MEMBER REGISTRATION & AUTHENTICATION

Registration and login establish identity and create the authenticated session used by protected operations. Passwords are stored as hashes, and the returned JWT is used by the frontend when calling protected APIs.

2. BOOK CATALOG MANAGEMENT

The catalog maintains title, author, ISBN, category and copy information. Book-level records form the central reference used by discovery and circulation.

3. CATALOG SEARCH & FILTERING

Search and filtering allow users to locate resources by title, author, ISBN or category and to identify whether usable copies are available.

4. BOOK ISSUE WORKFLOW

The issue workflow checks membership eligibility, active-loan limits and copy availability before creating a transaction and calculating its due date.

5. BOOK RETURN & FINE CALCULATION

Returns update the circulation record, derive overdue days, calculate the applicable fine and restore the book's availability.

6. RESERVATION / HOLD QUEUE

Members can join a waiting queue when a title is unavailable. Holds are ordered by request time and processed FIFO.

7. MEMBERSHIP PLANS & LIMITS

Plans define maximum active books, loan duration, overdue rate and maximum fine cap. This keeps policy configurable and reusable across circulation operations.

The first seven modules form the core circulation path. Authentication establishes identity, catalog management provides the resource, search helps the user locate it, and issue/return functions control its movement.

Reservations provide an alternative when no copy is available, while membership plans determine whether a borrowing action is permitted and how long the item may remain with the member.

Because these modules share state, they should not be evaluated independently. For example, a successful issue should affect both the transaction collection and book availability. A return should affect the transaction and may also affect the hold queue and fine information. This relationship is an important indicator of functional completeness.

The first group of modules forms a dependency chain. Authentication identifies the user; the catalogue provides resources; search helps locate a resource; membership policy determines eligibility; and circulation changes the resource state. Reservation becomes important when availability is zero. Because these modules are connected, testing them as an end-to-end sequence provides stronger evidence than demonstrating isolated forms.

For implementation quality, the module boundaries also help keep code understandable. Catalog logic should not contain password handling, and authentication should not calculate overdue fines. Separating concerns means each controller or service can be tested against a smaller set of responsibilities, while the overall application still behaves as one integrated system.

7. FUNCTIONAL MODULES – PART II

8. FINE PAYMENT TRACKING

Fine records maintain settlement status and payment information. Authorized staff can process approved waivers while retaining an auditable record.

9. OVERDUE NOTIFICATION RECORDS

Notification records support overdue reminders, book-availability messages and expiring-hold alerts. They create persistent application evidence even when external messaging is not configured.

10. INVENTORY & COPY MANAGEMENT

Inventory tracks total, available, lost and damaged copies. Adjustments record an action, quantity, reason and responsible actor so changes can be explained later.

11. MEMBER BORROWING HISTORY

Members can review previous circulation activity while staff can inspect operational history. This provides a persistent record rather than relying on the current-loan state alone.

12. LIBRARIAN / ADMIN REPORTS

Reports summarize overdue activity, borrowing patterns, fine information and inventory health. These views support operational decisions rather than only data entry.

13. ROLE-BASED ACCESS CONTROL

Role checks separate Member, Librarian and Admin operations. Authorization is enforced at the backend API boundary and not only through frontend navigation.

The remaining modules provide the administrative and accountability layer around circulation. Fine payment records answer whether a financial obligation has been settled, notification records preserve important event messages, and inventory logs explain changes to copy availability. Borrowing history and reports transform individual transactions into information that can be reviewed over time.

Role-based access control connects these modules to security. The member should see personal history and fines, while staff need broader operational information. Reports should not be treated as public endpoints because they aggregate institutional information. Protecting these modules at the backend keeps the application consistent even when requests are made directly through an API client.

The second group of modules provides the accountability surrounding circulation. Fine payments explain the financial state of a transaction, inventory logs explain changes to physical copies, notification records explain important user alerts, and reports summarize the resulting activity. These functions are valuable because a real library needs operational visibility after a transaction has occurred, not only at the moment of issue.

The modules also create opportunities for role-specific evaluation. A member can demonstrate personal history and fines, a librarian can demonstrate inventory and circulation, and an administrator can demonstrate institutional reports and policy management. The same database supports all three perspectives while authorization prevents one role from performing another role's operations.

8. TOOLS & TECHNOLOGIES USED

NODE.JS

Node.js provides the server-side JavaScript runtime. It executes the application, receives API requests, communicates with the database and runs the server-side business logic.

EXPRESS.JS

Express.js provides routing and middleware for the REST API. It organizes authentication, books, transactions, holds, fines, inventory and reporting endpoints.

MONGODB & MONGOOSE

MongoDB stores persistent application data as documents. Mongoose provides schemas, references, validation, indexes and database operations so the application can work with structured models.

JWT & BCRIPTJS

JWT is used for authenticated API sessions, while bcryptjs hashes passwords before storage. Together they provide the basic authentication mechanism for protected operations.

JOI, SECURITY & TESTING

Joi validates incoming data. Helmet-style headers, CORS and rate limiting provide additional API protection. Postman is used for manual REST testing, while Node.js test/integration workflows and MongoDB Memory Server can support repeatable backend verification.

FRONTEND STACK

HTML5, Bootstrap 5, CSS3 and vanilla JavaScript provide responsive role-specific pages. The frontend consumes the REST API and updates the interface according to authenticated user state.

The selected stack is cohesive because the same JavaScript ecosystem is used for server-side development and browser interaction. Express provides the HTTP layer, Mongoose provides structured access to MongoDB, and vanilla JavaScript communicates with the REST API. This reduces the number of unrelated technologies that students must integrate and makes the project easier to explain during the viva.

The security and testing tools are equally important to the technology stack. JWT and bcryptjs address authentication, Joi addresses input validation, security middleware protects the HTTP surface, and Postman provides repeatable API verification. These tools demonstrate that the project considers correctness and security alongside interface development.

Each technology contributes a specific layer to the implementation. Node.js and Express provide the execution and API environment, Mongoose translates application models into MongoDB operations, and the browser stack provides the visible interface. Security and validation libraries protect the boundary between the user and the business logic. Postman and backend testing tools then provide an independent way to verify that the implementation behaves correctly.

Using established technologies also improves the project's extensibility. A REST API can later support another frontend, while MongoDB can continue storing the same domain entities. Similarly, business services can be reused if the project adds barcode scanning, mobile access or automated notification jobs in a future version.

9. SYSTEM ARCHITECTURE

ARCHITECTURAL APPROACH

DLMS follows an MVC-style client-server architecture with a dedicated service/business layer. Presentation, HTTP routing, authentication, authorization, validation, business rules, persistence and database responsibilities are separated so that one layer does not become responsible for the entire application.

PRESENTATION LAYER

The browser contains the public landing experience, authentication screen, student/member dashboard, catalog, book details, reservation, history and fine views, together with librarian and administrator consoles. Role-aware navigation keeps each interface focused.

API & MIDDLEWARE

Express receives requests from the browser or Postman. Middleware verifies JWT tokens, checks roles, validates input with Joi and handles common security/error requirements before controllers process the request.

CONTROLLERS & SERVICES

Controllers coordinate HTTP requests and responses. Service functions apply circulation, fine, reservation, availability and inventory rules. This separation makes complex business behavior easier to test and explain.

DATA ACCESS & DATABASE

Mongoose models represent users, books, transactions, holds, payments, notifications, inventory logs and membership plans. MongoDB provides persistent storage and references connect entities with independent lifecycles.

The architecture is intentionally layered so that a change in one concern does not require rewriting every other concern. UI code is responsible for presentation, routes handle HTTP entry points, middleware handles cross-cutting checks, services implement business rules, and Mongoose handles persistence. This makes the application easier to debug because a problem can be traced to a specific layer.

The service layer is particularly valuable for library workflows. Fine calculation, borrowing limits, reservation order and inventory changes are business concepts rather than presentation concerns. Keeping them in the backend means the same rules apply whether the request originates from the browser or from Postman.

The architecture is designed around separation of concerns. Presentation code should not decide whether a member is eligible to borrow, and a database model should not decide how a button looks. Instead, each layer receives a well-defined responsibility. This makes failures easier to locate because an authentication problem, validation problem, business-rule problem and database problem can be distinguished during testing.

The service layer is particularly important for this project because library operations have side effects. A return may change a transaction, inventory availability, fine information and the reservation queue. Keeping these rules together makes the result more predictable than scattering the logic across several frontend handlers.

10. ARCHITECTURE FLOW & DESIGN BENEFITS

REQUEST FLOW

Browser / Frontend → Express Server → Routes & Middleware → JWT / RBAC / Joi → Controllers → Business Services → Mongoose Models → MongoDB → JSON Response.

EXAMPLE: BOOK ISSUE

When a member attempts to borrow a book, the frontend sends an authenticated request. Middleware validates the token and role. The controller invokes issue logic, which checks the member's plan and available copies, calculates the due date and creates the transaction. MongoDB is then updated and the response is returned to the UI.

EXAMPLE: BOOK RETURN

A librarian submits a return. The API verifies the staff role, loads the active transaction and determines whether the item is overdue. Fine logic uses the configured membership rate and cap. The transaction is updated, availability is restored and the reservation queue may be advanced.

MAINTAINABILITY

The layered design reduces coupling. A policy change can be handled in the service or membership-plan layer instead of being copied into every page. A new frontend can also consume the same REST API without changing the underlying database model.

VIVA VALUE

The architecture provides a simple explanation path during evaluation: show the screen, identify the API endpoint, explain the middleware check, describe the business rule, identify the affected MongoDB collection and show the resulting UI state.

A useful way to validate the architecture is to trace a complete request. The browser initiates an action, the API receives it, middleware checks identity and permissions, validation checks the data, the controller invokes business logic, the service reads or changes the relevant documents, and the response returns to the interface. Every step has a clear responsibility.

This approach also improves maintainability and testing. A business rule can be tested without rendering a page, while the frontend can be tested against known API responses. If the project is extended later, new interfaces can consume the same API contract, and new integrations can reuse the same business services.

The request flow can be used as a technical explanation for almost every feature in the application. The evaluator can select a visible action and ask what happens next. The answer follows the same pattern: frontend request, Express route, middleware, controller/service, Mongoose operation and JSON response. This consistent flow reduces the complexity of explaining the project because different features follow a common architectural pattern.

The architecture also supports independent testing. API tests can call endpoints directly, validation can be tested with malformed data, authorization can be tested using different tokens, and database behavior can be verified using controlled test data. The frontend then becomes one client of the same tested backend rather than the only place where correctness is demonstrated.

11. SYSTEM WORKFLOW

1. PUBLIC DISCOVERY

The visitor starts on the landing page. The interface explains the digital-library purpose, exposes discovery options and provides a clear route toward authentication and catalog access.

2. AUTHENTICATION

The user enters credentials. The backend validates the account and returns a JWT. The frontend stores the authenticated state and loads the correct role-specific navigation.

3. CATALOG DISCOVERY

A student/member searches or filters the academic catalog and opens a book detail page. Availability and bibliographic information are visible before a circulation request is attempted.

4. BORROWING

The issue workflow checks plan eligibility, active-loan limits and availability. A transaction is created and a due date is calculated from the membership plan.

5. RETURN & FINE

The librarian processes the return. Overdue days and fine are calculated, the transaction is updated and the copy becomes available for future circulation.

6. RESERVATION

If the title is unavailable, the member can join the hold queue. Returned copies can advance the earliest eligible hold and create a notification record.

7. REPORTING

Administrative and staff views summarize circulation, fines and inventory so that the library can monitor operational activity.

The workflow deliberately begins before the user logs in because a real library application has both public and protected areas. The landing page establishes the service and discovery path. Authentication then creates the trusted user context needed for personal data. After that, the member can move naturally from catalog discovery to book details and circulation.

The workflow also illustrates how the staff side completes the lifecycle. A member action creates a request, but staff may be responsible for operational processing. The resulting database state is then visible again to the member through the dashboard, history, reservation or fine views. This feedback loop is what makes the system useful for continuous library operations.

The user journey is deliberately ordered from simple to complex. Public discovery requires no account, authentication establishes identity, catalog discovery identifies a resource, and circulation introduces state changes. If the resource is unavailable, the reservation workflow provides a second path. Once the item is returned, the system can complete the cycle through inventory, fine and notification updates. This sequence provides a natural script for the project demonstration.

The workflow also creates feedback between roles. A member's action can create a record that later appears in a librarian's console. A librarian's return action can change what the member sees in the dashboard and can make a reserved title available. This two-way interaction demonstrates that the application is backed by shared persistent state rather than hard-coded screen values.

12. BORROWING, RETURN & FINE RULES

BORROWING RULES

A member cannot exceed the active borrowing quota defined by the membership plan. A title cannot be issued when no eligible copy is available. Once approved, the due date is derived from the plan's loan duration and the transaction becomes part of the member's borrowing history.

RETURN RULES

A return must reference an active transaction. The system compares the return date with the due date, derives overdue days and calculates the fine according to the member's daily rate and maximum cap. The book's availability is then restored according to the recorded inventory state.

FINE LOGIC

Fine amount is based on overdue days multiplied by the applicable daily rate and limited by the configured maximum. This policy-driven calculation prevents inconsistent manual values and allows Student and Faculty plans to use different rates.

PAYMENT & WAIVER

Outstanding fines can be settled through the fine-payment workflow. Authorized staff can record an approved waiver. Ordinary members cannot directly remove or alter their own fine through the API because the operation is role restricted.

AUDITABILITY

The transaction, payment and inventory records together provide a traceable chain. This is important during evaluation because it allows a staff action to be explained as a sequence of controlled data changes.

Borrowing and return operations are state transitions, not isolated form submissions. The issue operation creates a new active transaction and changes availability. The return operation closes that transaction and can create a financial consequence if the due date has passed. Treating these effects as one business operation reduces the chance that the database will contain a transaction saying ACTIVE while the corresponding book is shown as freely available.

The fine policy is intentionally configuration-driven. A membership plan can specify the daily rate and maximum cap, allowing the same return logic to serve multiple member categories. This also makes the behavior easier to demonstrate: the evaluator can identify the member category, observe the due date, calculate the overdue period and compare the result with the recorded fine.

The circulation rules are important because they prevent inconsistent states. If the application allowed a member to borrow without checking availability, the same copy could appear to be issued to multiple people. If a return did not restore availability, the catalogue would remain inaccurate. If fine calculation were left to the frontend, a user could potentially manipulate the value. Placing these decisions in the backend keeps the source of truth under server control.

The policy-driven fine model also improves transparency. Staff can explain a fine by identifying the due date, overdue days, membership rate and maximum cap. Members can then see the resulting obligation in the portal. This is more reliable than entering a manually calculated number into a form and makes the rule easier to test through API calls.

13. RESERVATION, INVENTORY & NOTIFICATIONS

FIFO RESERVATION QUEUE

When a title is unavailable, the member can create a hold. The requestedAt value establishes the queue order. The earliest eligible member is promoted when a copy becomes available, ensuring that reservations are handled fairly.

HOLD LIFECYCLE

A hold can be created, remain waiting, become available for collection, expire or be cancelled. The workflow can move to the next waiting member when the current opportunity expires.

INVENTORY ACCOUNTABILITY

Inventory is more than a copy count. The system needs to distinguish usable copies from copies that are lost or damaged. Additions, losses, damage, restoration and corrections can therefore be represented as explicit audit actions.

NOTIFICATION RECORDS

Notification records support overdue reminders, availability messages and expiring-hold alerts. Even without external email/SMS delivery, these records provide application-level evidence that the event occurred.

INTEGRATED EFFECT

A return may change several modules at once: the transaction becomes returned, availability increases, a fine may be generated, a waiting member may be promoted and a notification may be created. This multi-step effect demonstrates real workflow integration.

Reservation handling is important because a library must provide a fair mechanism when demand exceeds supply. FIFO ordering means the system can explain why a particular member was served first. The queue is therefore not merely a list on the screen; it represents a business rule that must be respected by the backend whenever a copy becomes available.

Inventory and notification records provide operational context around the same event. A returned copy increases availability, an eligible hold may become ready, and a notification may be created. Recording these effects makes the system more transparent and gives staff information that can be used to explain what happened to a resource.

The reservation mechanism provides fairness under limited supply. If several members request the same title, the queue order gives the system a deterministic way to decide who should be served next. This is a clear example of why the project needs business logic: a simple database insert is not enough unless later processing respects the order established by earlier requests.

Inventory records provide the physical-resource perspective, while notifications provide the user-communication perspective. Together they show how one event can have multiple consequences. A return is therefore not just a transaction update; it can restore a copy, advance a hold, create an alert and change what staff see in reports.

14. MEMBERSHIP PLANS & REPORTING

MEMBERSHIP POLICY

Membership plans define maximum active books, loan duration, daily overdue rate and maximum fine cap. Student and Faculty categories can therefore be governed by different policies without duplicating circulation logic.

REPORTING PURPOSE

Reports provide an operational view of the library. Instead of opening individual transactions, staff can review overdue activity, borrowing patterns, fine information and inventory health in summary form.

MEMBER DASHBOARD

The student/member dashboard focuses on personal activity: current loans, due-soon status, holds, fines, recommendations and access to the academic catalog. It reduces the amount of information shown at one time while still giving the user an overview.

STAFF DASHBOARD

Staff-oriented screens provide more operational information. Circulation records, inventory actions, fine information and administrative controls are visible according to the user's role.

ADMINISTRATION

The administrative console is intended for institutional management. It demonstrates that the same backend can serve multiple interfaces while maintaining separate permissions and responsibilities.

Membership plans make policy explicit. Instead of embedding values such as maximum loans or fine rates throughout the application, the system can retrieve them as part of the member's policy. This approach is easier to maintain and makes differences between Student and Faculty categories visible in the data model.

Reporting is the final aggregation layer. Individual transactions are useful for one member or one book, but library staff also need to know which books are borrowed frequently, how many loans are overdue and whether inventory is healthy. Reports therefore turn transactional records into operational information that can support decisions.

Keeping membership rules in data makes the application adaptable. If the university changes the number of books a student may borrow or modifies the overdue rate, the concept of the workflow remains unchanged. The circulation service reads the policy and applies it. This reduces the need for repeated code changes and makes the design more maintainable.

Reports provide value because raw database records are not always convenient for operational decisions. Staff may want to know which books are most frequently borrowed, which loans are overdue or how inventory is distributed. Aggregating the existing records into focused reports gives the library a higher-level view without changing the underlying transactional data.

15. DATABASE DESIGN

COLLECTIONS

The main MongoDB collections are users, books, transactions, holds, finePayments, notifications, inventoryLogs and membershipPlans. Each collection represents a meaningful domain entity rather than storing the entire application in one document.

USERS

The users collection stores identity, role and member category information. Authentication data is kept separate from book and transaction records so that authorization can be applied consistently.

BOOKS

Books store bibliographic information and copy availability. Catalog queries can operate on title, author, ISBN and category while circulation operations use the availability state.

TRANSACTIONS

Transactions connect a member and book with issue date, due date, return date, fine and status. This collection forms the core of borrowing history and overdue reporting.

HOLDS, PAYMENTS & LOGS

Holds connect members to requested books and preserve queue order. Fine payments connect financial settlement to the originating transaction. Inventory logs connect copy changes to a responsible actor and reason.

MEMBERSHIP PLANS

Plans hold policy values such as maxBooks, loanDuration, overdueRate and maxFineCap. This design allows circulation behavior to depend on configuration rather than duplicated constants.

The database is organized around entities that have different lifecycles. Users remain in the system even when they have no active loan; books remain in the catalog after being returned; transactions preserve historical circulation; holds represent temporary demand; and inventory logs preserve a history of physical-resource changes. This separation makes the data easier to query and audit.

References are used where an entity needs to point to another independently managed record. For example, a transaction can reference both a member and a book. A fine payment can reference the transaction that generated it. An inventory log can reference the book and actor involved. These relationships allow the system to preserve context without duplicating every field.

The collection design separates current state from historical evidence. A book stores catalogue and availability information, while a transaction preserves a particular borrowing event. This means the system does not lose historical information simply because a book is returned. Similarly, an inventory log records why a copy count changed, while the book record stores the current result.

This structure also supports future extensions. Additional attributes can be added to individual domain collections without forcing the entire application into one large document. References allow the application to retrieve related information when needed and keep independently managed entities logically separate.

16. DATABASE INDEXING & RELATIONSHIPS

INDEXING STRATEGY

Indexes should follow real query patterns. A user email index speeds authentication and duplicate checking; title and author indexes support catalog discovery; memberId and bookId indexes support history; bookId plus requestedAt supports FIFO hold retrieval.

RELATIONSHIPS

A member can have many transactions and holds. A book can appear in many transactions and holds over time. A fine payment references its originating transaction. Inventory logs reference the affected book and the responsible staff actor.

CONSISTENCY

The most important consistency rule is the relationship between transaction state and availability. Issue reduces available copies, return restores availability, while loss and damage can reduce usable stock. Backend rules should be responsible for these changes.

AUDIT DESIGN

Inventory adjustments require a reason and actor, which makes the change explainable later. Circulation history and payment records provide similar traceability for borrowing and financial operations.

DIAGRAM FOR REPORT

The final submission may include the project's exact ER/collection diagram here. The report should show users, books, transactions, holds, payments, notifications, inventory logs and membership plans with their key references.

Indexing should be guided by actual application behavior. Authentication repeatedly looks up email addresses; catalog users search titles and authors; members request their history; staff retrieve active transactions; and reservation processing needs the earliest waiting member. Indexes on these fields reduce the amount of data MongoDB needs to scan for common operations.

The relationship model also supports reporting and auditing. Because transactions reference members and books, the application can reconstruct a member's history or a title's circulation history. Because payments reference transactions, financial settlement remains connected to the original borrowing event. Because inventory logs reference actors, staff changes can be explained later.

Indexing is useful only when it supports real access patterns. Authentication repeatedly searches by email, catalogue discovery searches by bibliographic fields, member history uses member identifiers, and hold processing needs the earliest request for a book. Selecting indexes around these operations provides a practical performance strategy rather than adding indexes without a purpose.

Relationships also improve traceability. From a transaction, the system can identify the member and book. From a payment, it can identify the originating transaction. From an inventory log, it can identify the book and responsible actor. These links allow staff to explain records in context and make the database more useful for reporting and auditing.

17. REST API DOCUMENTATION

AUTHENTICATION

POST /api/auth/register registers a library member. POST /api/auth/login authenticates the account and returns a JWT. Protected endpoints use the token in the Authorization header.

CATALOG

GET /api/books/search provides catalog discovery. Book-management endpoints can be restricted to staff roles when creating or updating catalog records.

CIRCULATION

POST /api/transactions/issue creates an issue transaction after eligibility and availability checks. PUT /api/transactions/:id/return processes a return and fine calculation.

RESERVATION & HISTORY

POST /api/holds creates a reservation. GET /api/members/:id/history exposes borrowing history to permitted users.

FINES & REPORTS

POST /api/fines/:transactionId/pay records fine settlement. GET /api/admin/reports/overdue and GET /api/admin/reports/inventory provide operational reporting for authorized administrative users.

SAMPLE ISSUE REQUEST

```
POST /api/transactions/issue
Authorization: Bearer <JWT>
{ "bookId": "64f1...d1", "memberId": "64f1...d2" }
```

SUCCESS RESPONSE

```
HTTP 201 Created
{ "success": true, "message": "Book issued successfully", "data": { "transactionId": "64f1...d3", "status": "ACTIVE" } }
```

The REST API provides a stable contract between the frontend and backend. A browser action should correspond to a defined HTTP method and endpoint, and the endpoint should return a predictable JSON structure. This separation is useful because Postman can reproduce the same request without the browser, allowing the evaluator to verify backend behavior independently.

Protected endpoints also demonstrate the value of authentication headers. The JWT travels with the request, middleware verifies it, and the endpoint then performs the permitted operation. Invalid or missing credentials are rejected before sensitive business logic is executed. This design keeps the API responsible for enforcing the application's security boundary.

The REST API is the technical bridge between the browser and the domain logic. A consistent endpoint structure allows the frontend to remain focused on presentation while the backend remains responsible for validation and business decisions. It also makes the project easier to test because Postman can reproduce the same HTTP request without loading the complete user interface.

Structured responses are important for frontend reliability. A successful response should provide a clear success state and relevant data, while a failure should provide a predictable error structure. This lets the frontend show useful feedback without needing to interpret raw database errors or server exceptions.

18. VALIDATION, SECURITY & ERROR HANDLING

VALIDATION

Joi/Express middleware validates incoming request bodies and parameters before business logic runs. This prevents malformed data from reaching controllers and makes validation behavior consistent across endpoints.

AUTHENTICATION & PASSWORD SECURITY

JWT tokens identify authenticated sessions. bcryptjs hashes passwords before storage, preventing the database from containing plaintext credentials.

AUTHORIZATION

Role checks protect staff and administrative functions. A valid token alone is not sufficient; the user's role must also satisfy the endpoint requirement.

API SECURITY

Environment variables are used for database configuration and JWT secrets. CORS, Helmet-style security controls and rate limiting provide additional protection at the HTTP/API boundary.

CENTRALIZED ERRORS

The application can return consistent JSON errors for invalid IDs, missing fields, duplicate records, authentication failures, authorization failures and business-rule conflicts. This prevents raw server exceptions from leaking into the frontend.

MANDATORY NEGATIVE CASES

401 Unauthorized: no valid token. 403 Forbidden: wrong role. 400/409: invalid input or business conflict. 404 Not Found: missing resource. These cases are important evidence that the backend actively enforces rules.

Security is treated as a layered responsibility. Password hashing protects stored credentials, JWT protects authenticated sessions, RBAC protects privileged operations, Joi reduces malformed input, security middleware protects the HTTP layer, and centralized errors keep failures predictable. No single mechanism is expected to solve every security concern.

Negative testing is especially important for this type of application. The most useful tests are often the requests that should fail: a missing token, a wrong role, an unavailable copy, an invalid identifier or a request that violates a borrowing limit. Demonstrating these cases gives stronger evidence that the backend contains real business logic.

The security design follows a defense-in-depth approach. Authentication verifies identity, authorization verifies permission, validation verifies data shape, business rules verify domain correctness, and centralized error handling controls how failures are returned. This layered approach means that an invalid request should be rejected at the earliest appropriate point rather than being allowed to reach the database unchecked.

For the viva, negative cases are especially useful because they reveal the quality of the backend. Demonstrating a missing token, wrong role, unavailable copy or invalid identifier shows that the system does not simply accept every request. It also gives a clear explanation of why middleware and business services are necessary in a real application.

19. POSTMAN TESTING & RESULTS

TESTING APPROACH

The Postman collection is used to verify both successful operations and failure cases. Testing the API independently from the browser is important because it demonstrates that security and business rules are enforced at the backend boundary.

AUTHENTICATION TESTS

Valid registration and login should return successful responses and a JWT. Calling a protected route without the token should return 401 rather than executing the operation.

AUTHORIZATION TESTS

A member attempting a staff-only endpoint should receive 403. This verifies that the backend does not trust frontend navigation or hidden buttons as a security mechanism.

BUSINESS RULE TESTS

Issue tests verify eligibility and availability. Out-of-stock or loan-limit cases should be rejected. Return tests verify fine calculation and inventory restoration. Reservation tests verify queue creation and progression.

INVENTORY & REPORT TESTS

Inventory changes should create auditable records. Report endpoints should return the expected summaries for authorized users. Invalid IDs should return 404 without crashing the server.

TESTING SUPPORT

The documented project also supports backend/integration workflows and MongoDB Memory Server so repeatable tests can be executed without depending on a permanent external test database.

Postman testing should follow the same sequence used in the browser demonstration. Start with authentication, then test catalog access, issue, return, reservation, fines and reports. After the successful paths are verified, repeat selected requests with invalid tokens, wrong roles, missing fields and conflicting states. This produces evidence for both functional correctness and defensive behavior.

A useful result is not only a 200-series response. The expected status code and response body should match the business meaning of the request. A rejected issue because no copy is available is a successful security/business-rule test even though the HTTP response is an error class. This distinction is important when presenting test results.

Postman testing can be organized as a repeatable demonstration sequence. Authentication establishes the token, successful functional requests verify the main workflow, and negative requests verify the protection around it. Keeping these cases together in a collection also makes it easier for another evaluator or team member to reproduce the tests without relying on the browser state.

The most meaningful test results connect an HTTP response with a business expectation. For example, a rejected out-of-stock issue is evidence that the availability rule works; a 403 response for a member calling a staff endpoint proves RBAC; and a 404 response for an invalid ID proves that the server handles missing resources cleanly. These are functional results even though some expected results are error responses.

20. FRONTEND, SCREENSHOT EVIDENCE & CONCLUSION

FRONTEND DESIGN

The frontend follows a clear user journey: public landing page, authentication, student/member dashboard, academic catalog, book details, reservations, borrowing history and fines, followed by staff and administrator consoles. This structure mirrors how an evaluator can experience the system during a demonstration.

SCREENSHOT ORDER

The following evidence is intentionally arranged in the requested order: Landing Page → Login Page → Student/Member Dashboard → Student/Member Components → Admin Console → Staff Console. This removes the earlier problem of screenshots appearing in a random order.

LIMITATIONS

The current implementation is suitable for academic demonstration and controlled deployment. Production expansion would require additional infrastructure and external integrations for real payment processing, messaging, scanning devices, scheduled background jobs, monitoring and stronger operational security.

FUTURE SCOPE

Possible improvements include real payment gateways, email/SMS delivery, barcode/RFID issue-return, multi-branch inventory, scheduled notification jobs, advanced search indexing, richer analytics, expanded frontend testing, centralized logs, observability, token revocation and stronger backup/audit controls.

CONCLUSION

DLMS demonstrates a complete academic-library workflow using Node.js, Express.js, MongoDB, Mongoose, JWT, bcryptjs, Joi, Bootstrap and Postman. It combines catalog discovery with circulation, reservations, fines, inventory, membership policy, notifications and reporting. The layered backend and role-aware frontend provide a practical system that can be explained from the user interface down to the database.

SCREENSHOTS & IMPLEMENTATION EVIDENCE

Figure 1. Landing Page – Library discovery and search entry

These screens establish the public entry point and academic-library identity before authentication.

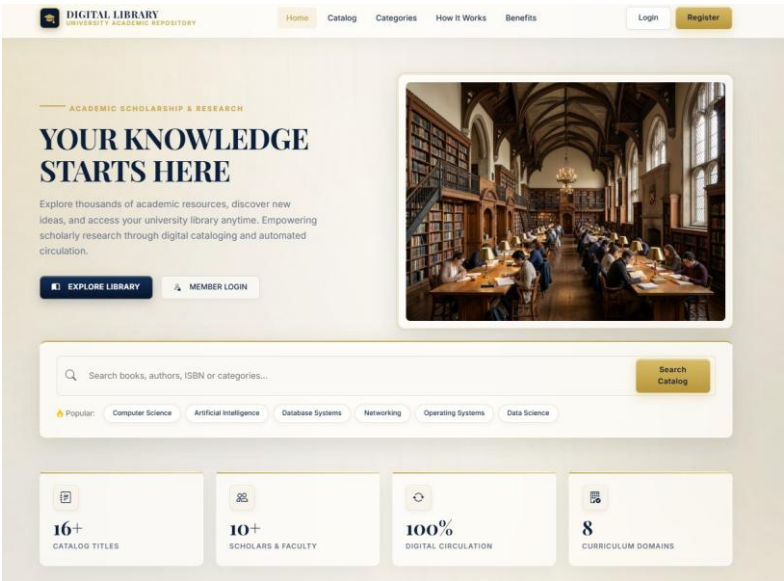


Figure 1. Landing Page – Library discovery and search entry

EVIDENCE INTERPRETATION

The screen demonstrates the corresponding role and workflow described in the report. During evaluation, the evaluator can perform the visible action and then verify the related API/database change. This connects the screenshot evidence to the actual implementation instead of treating the image as standalone decoration.

Figure 2. Landing Page – Browse by academic subject area

These screens establish the public entry point and academic-library identity before authentication.

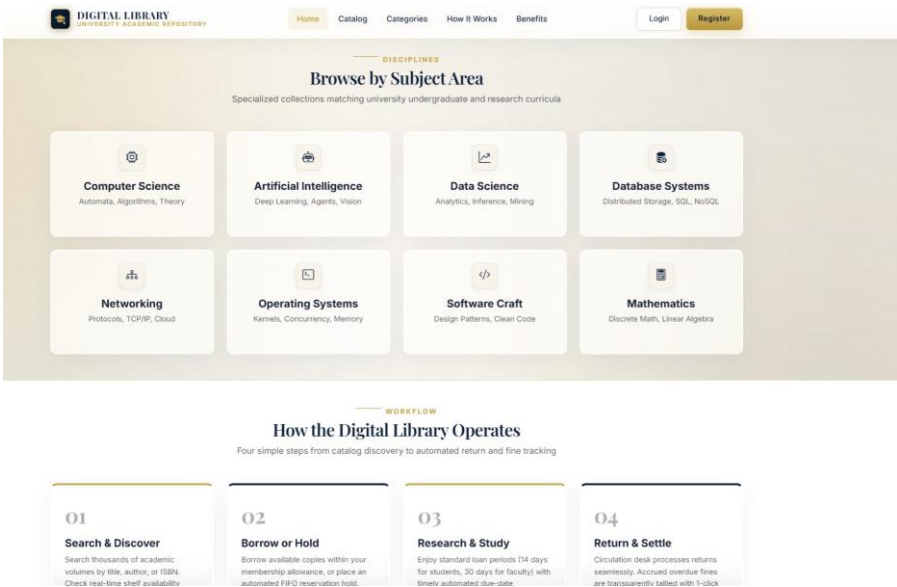


Figure 2. Landing Page – Browse by academic subject area

EVIDENCE INTERPRETATION

The screen demonstrates the corresponding role and workflow described in the report. During evaluation, the evaluator can perform the visible action and then verify the related API/database change. This connects the screenshot evidence to the actual implementation instead of treating the image as standalone decoration.

SCREENSHOTS & IMPLEMENTATION EVIDENCE

Figure 3. Landing Page – Workflow, benefits and footer

The login screen acts as the authentication boundary and the first protected step in the user journey.

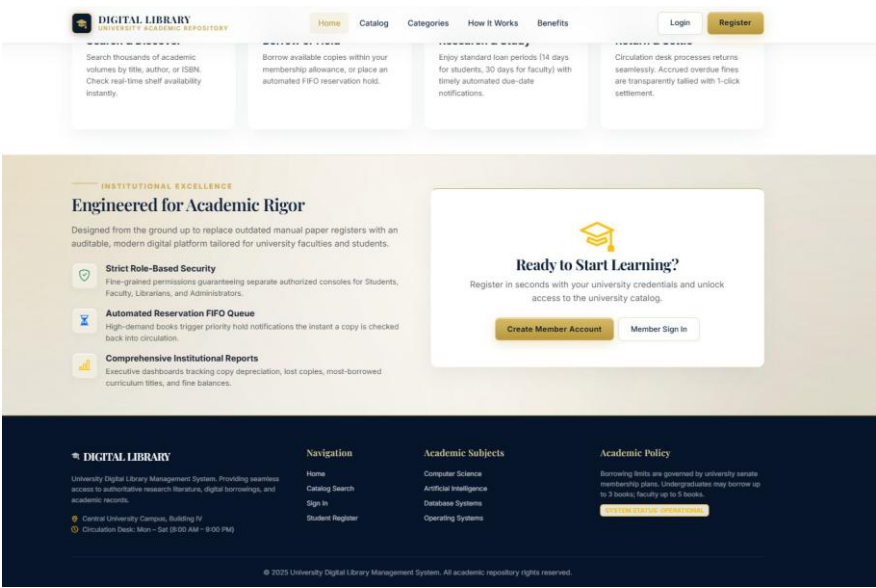


Figure 3. Landing Page – Workflow, benefits and footer

EVIDENCE INTERPRETATION

The screen demonstrates the corresponding role and workflow described in the report. During evaluation, the evaluator can perform the visible action and then verify the related API/database change. This connects the screenshot evidence to the actual implementation instead of treating the image as standalone decoration.

Figure 4. Login Page – Institutional role-aware authentication

The login screen acts as the authentication boundary and the first protected step in the user journey.

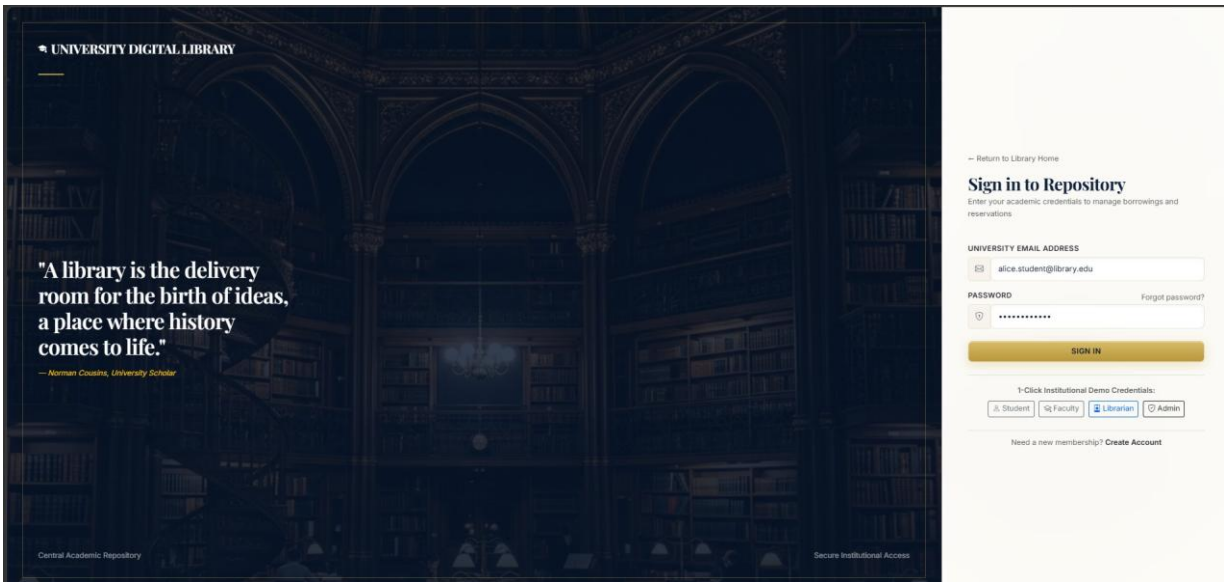


Figure 4. Login Page – Institutional role-aware authentication

EVIDENCE INTERPRETATION

The screen demonstrates the corresponding role and workflow described in the report. During evaluation, the evaluator can perform the visible action and then verify the related API/database change. This connects the screenshot evidence to the actual implementation instead of treating the image as standalone decoration.

SCREENSHOTS & IMPLEMENTATION EVIDENCE

Figure 5. Student/Member Dashboard – Scholar overview

The dashboard provides an overview of personal library activity and then transitions into the academic catalog.

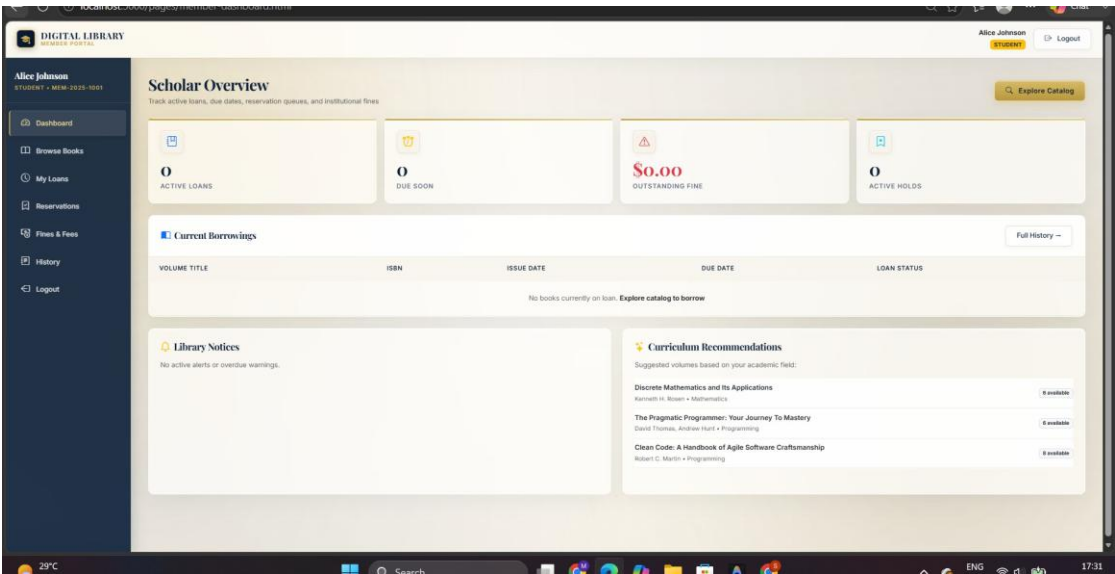


Figure 5. Student/Member Dashboard – Scholar overview

EVIDENCE INTERPRETATION

The screen demonstrates the corresponding role and workflow described in the report. During evaluation, the evaluator can perform the visible action and then verify the related API/database change. This connects the screenshot evidence to the actual implementation instead of treating the image as standalone decoration.

Figure 6. Student/Member Portal – Academic catalog

The dashboard provides an overview of personal library activity and then transitions into the academic catalog.

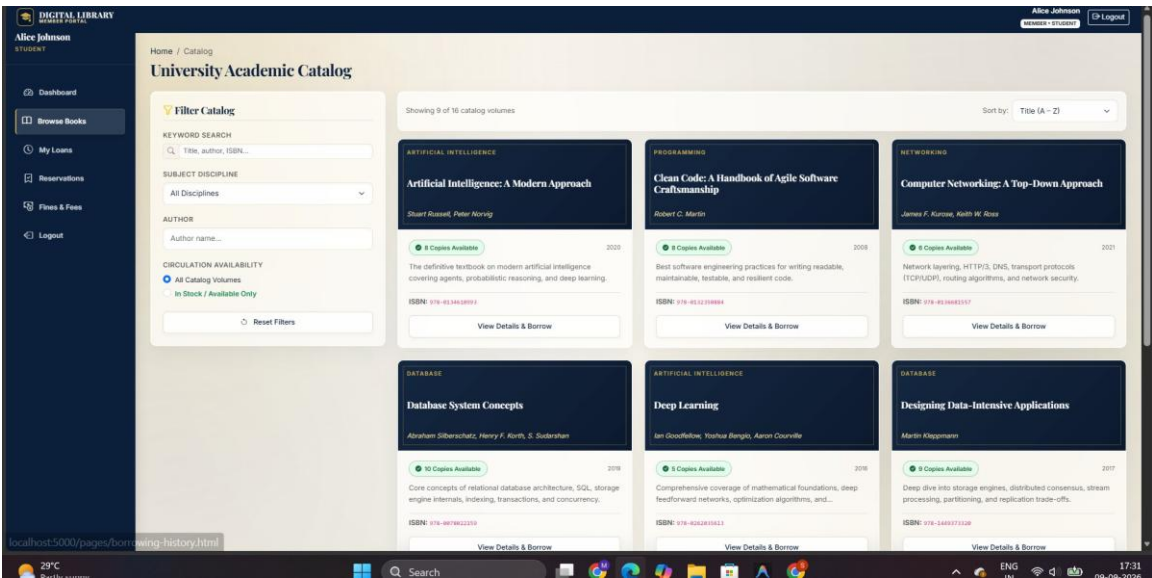


Figure 6. Student/Member Portal – Academic catalog

EVIDENCE INTERPRETATION

The screen demonstrates the corresponding role and workflow described in the report. During evaluation, the evaluator can perform the visible action and then verify the related API/database change. This connects the screenshot evidence to the actual implementation instead of treating the image as standalone decoration.

SCREENSHOTS & IMPLEMENTATION EVIDENCE

Figure 7. Student/Member Portal – Book details and borrowing action

Book details, holds and borrowing history show the main member workflows after catalog discovery.

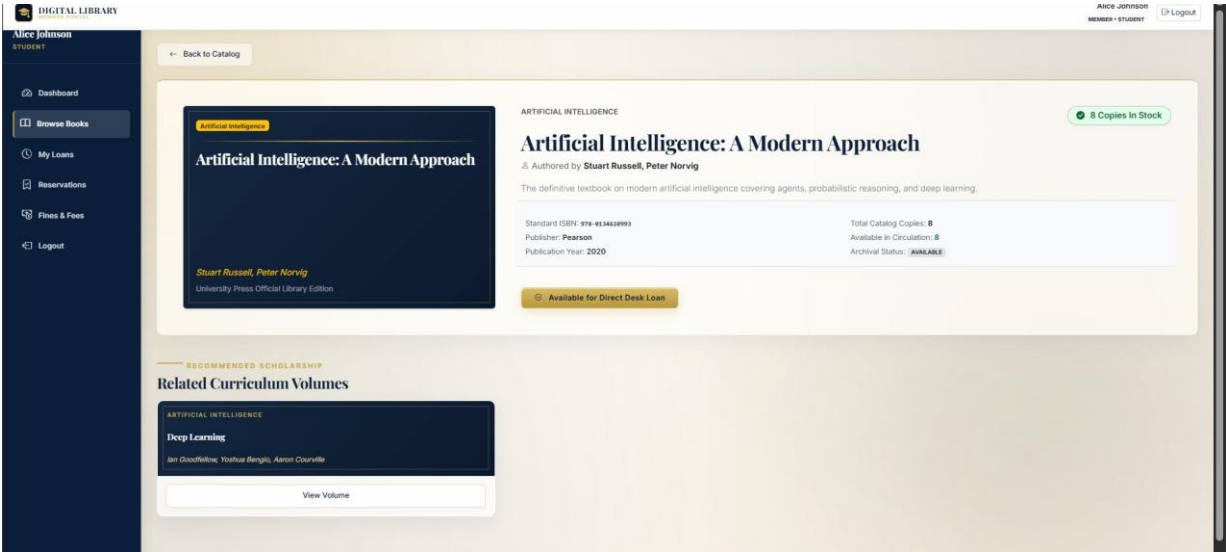


Figure 7. Student/Member Portal – Book details and borrowing action

EVIDENCE INTERPRETATION

The screen demonstrates the corresponding role and workflow described in the report. During evaluation, the evaluator can perform the visible action and then verify the related API/database change. This connects the screenshot evidence to the actual implementation instead of treating the image as standalone decoration.

Figure 8. Student/Member Portal – Reservation and hold queue

Book details, holds and borrowing history show the main member workflows after catalog discovery.

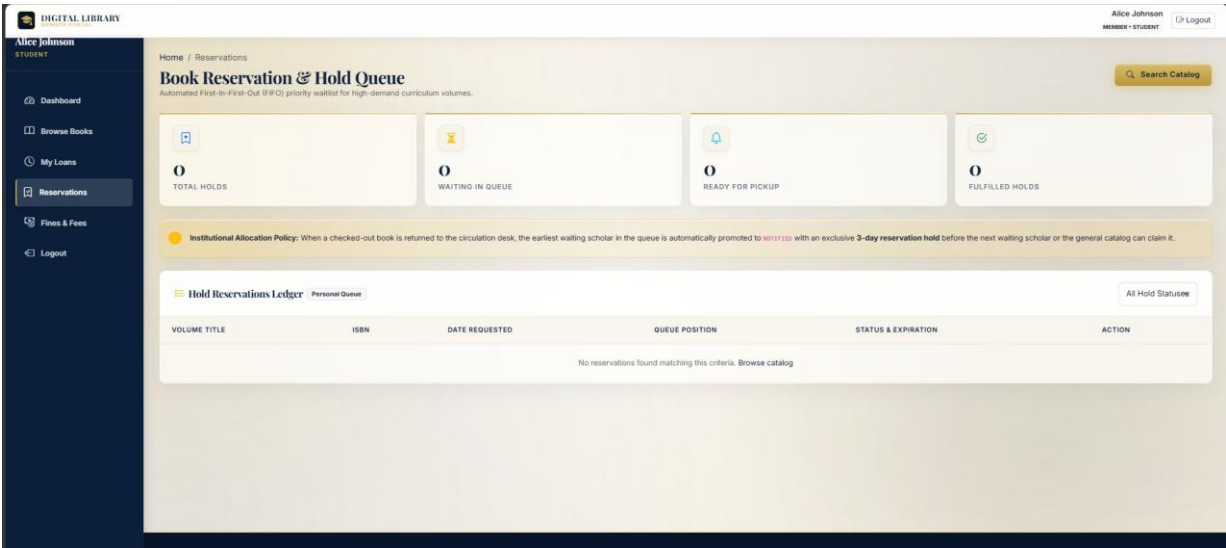


Figure 8. Student/Member Portal – Reservation and hold queue

EVIDENCE INTERPRETATION

The screen demonstrates the corresponding role and workflow described in the report. During evaluation, the evaluator can perform the visible action and then verify the related API/database change. This connects the screenshot evidence to the actual implementation instead of treating the image as standalone decoration.

SCREENSHOTS & IMPLEMENTATION EVIDENCE

Figure 9. Student/Member Portal – Borrowing history

The fine screen completes the personal circulation view by exposing outstanding settlement information.

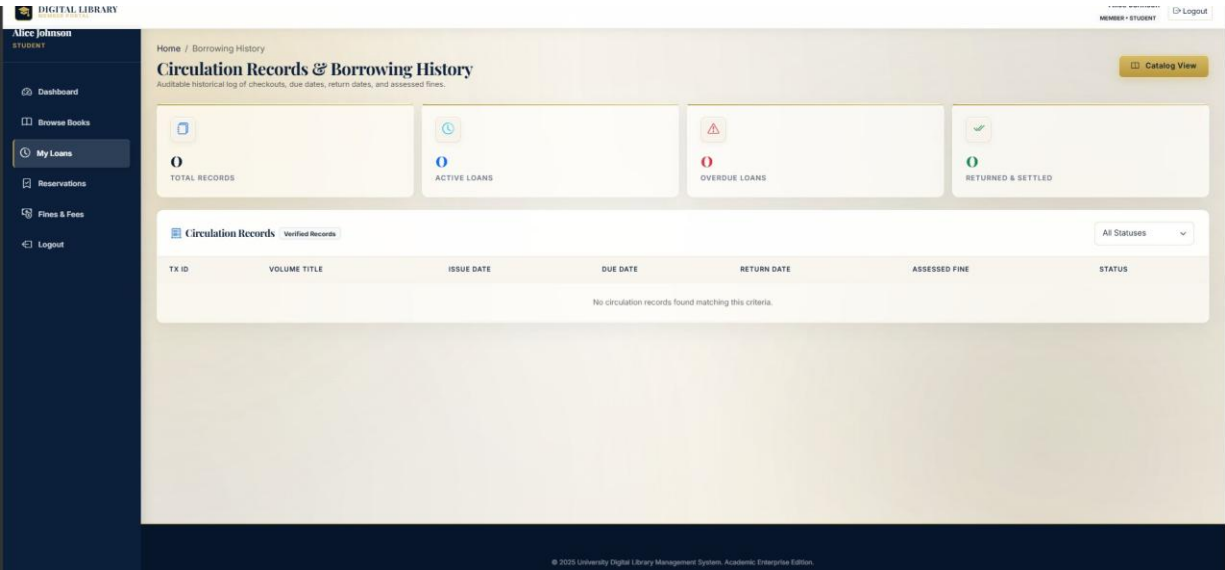


Figure 9. Student/Member Portal – Borrowing history

EVIDENCE INTERPRETATION

The screen demonstrates the corresponding role and workflow described in the report. During evaluation, the evaluator can perform the visible action and then verify the related API/database change. This connects the screenshot evidence to the actual implementation instead of treating the image as standalone decoration.

Figure 10. Student/Member Portal – Fines and settlements

The fine screen completes the personal circulation view by exposing outstanding settlement information.

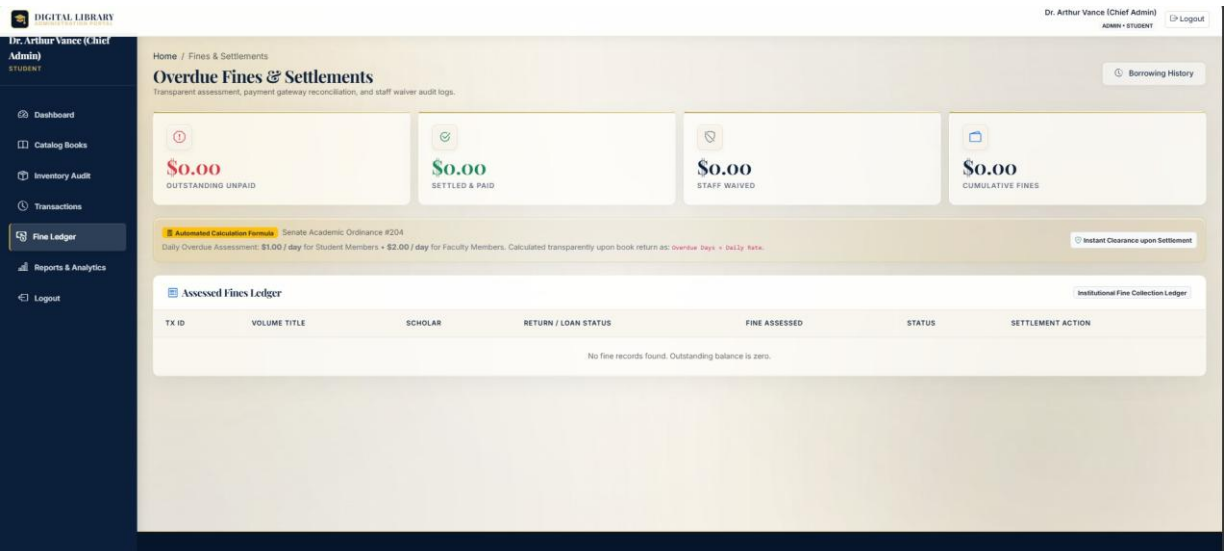


Figure 10. Student/Member Portal – Fines and settlements

EVIDENCE INTERPRETATION

The screen demonstrates the corresponding role and workflow described in the report. During evaluation, the evaluator can perform the visible action and then verify the related API/database change. This connects the screenshot evidence to the actual implementation instead of treating the image as standalone decoration.

SCREENSHOTS & IMPLEMENTATION EVIDENCE

Figure 11. Admin Console – Institutional administration

The administrative and inventory screens demonstrate institutional control and copy-level accountability.

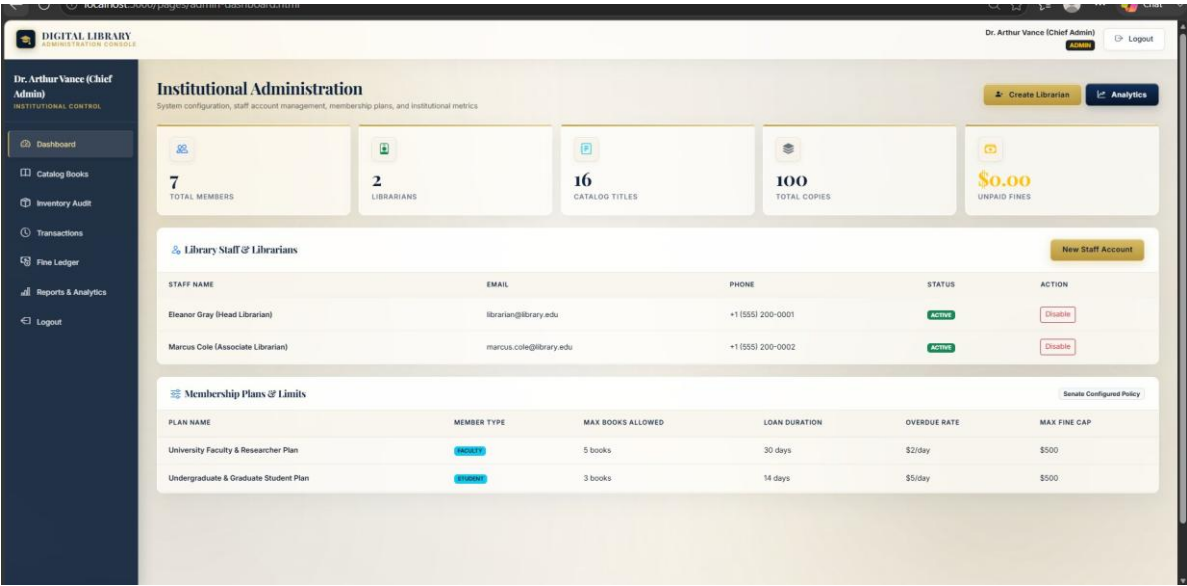


Figure 11. Admin Console – Institutional administration

EVIDENCE INTERPRETATION

The screen demonstrates the corresponding role and workflow described in the report. During evaluation, the evaluator can perform the visible action and then verify the related API/database change. This connects the screenshot evidence to the actual implementation instead of treating the image as standalone decoration.

Figure 12. Admin Console – Library inventory and copy ledger

The administrative and inventory screens demonstrate institutional control and copy-level accountability.

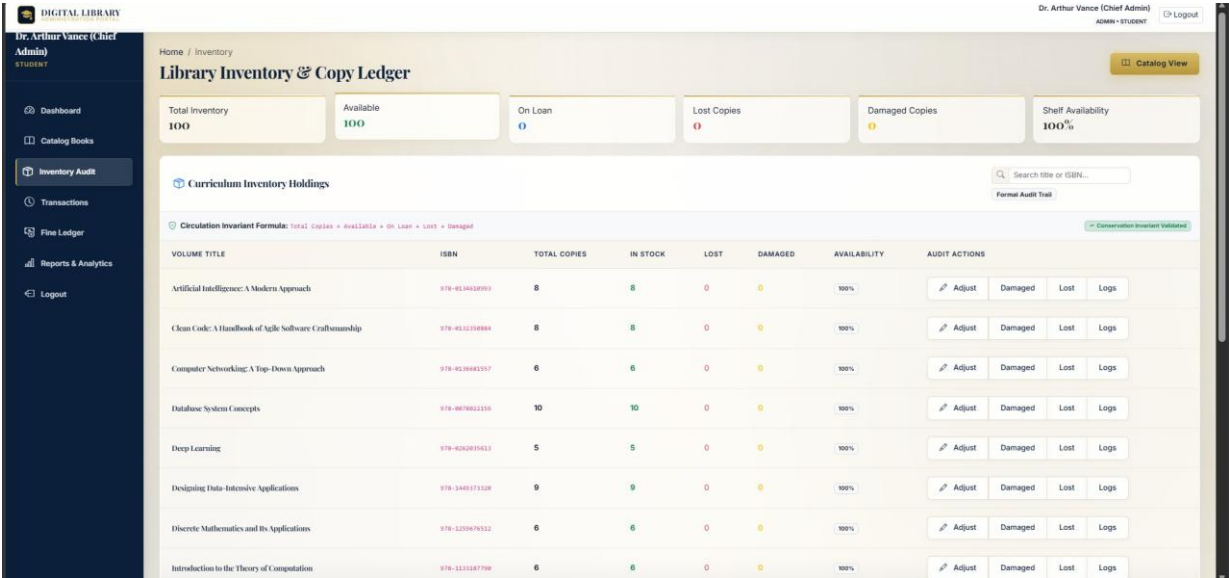


Figure 12. Admin Console – Library inventory and copy ledger

EVIDENCE INTERPRETATION

The screen demonstrates the corresponding role and workflow described in the report. During evaluation, the evaluator can perform the visible action and then verify the related API/database change. This connects the screenshot evidence to the actual implementation instead of treating the image as standalone decoration.

SCREENSHOTS & IMPLEMENTATION EVIDENCE

Figure 13. Staff Console – Circulation records and borrowing history

The final staff view demonstrates the operational perspective on circulation and complements the student/member workflow.

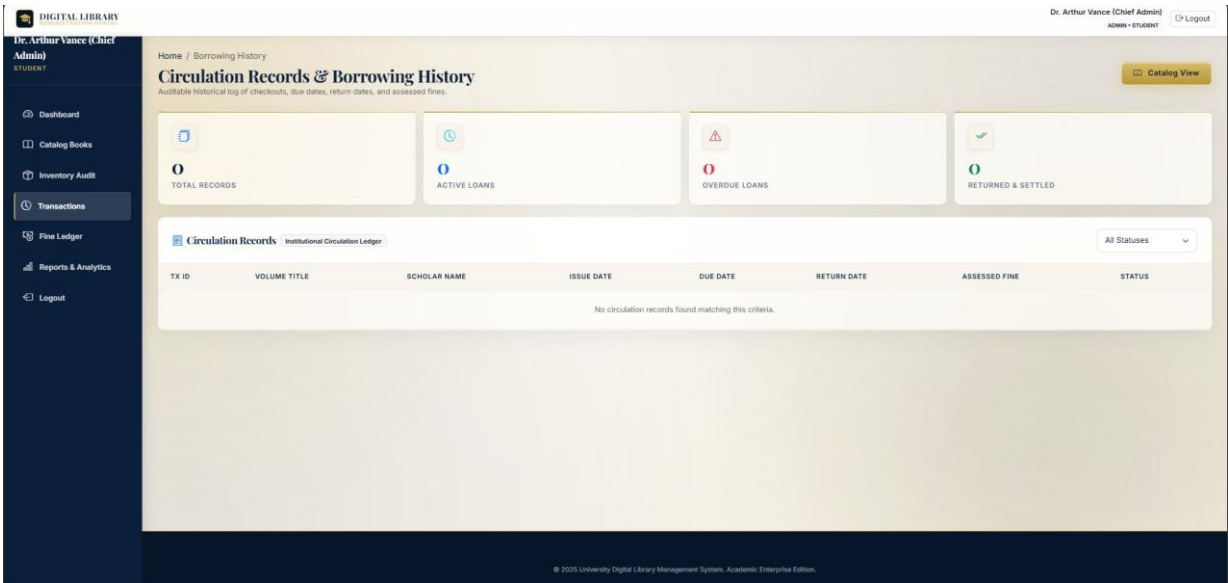


Figure 13. Staff Console – Circulation records and borrowing history

EVIDENCE INTERPRETATION

The screen demonstrates the corresponding role and workflow described in the report. During evaluation, the evaluator can perform the visible action and then verify the related API/database change. This connects the screenshot evidence to the actual implementation instead of treating the image as standalone decoration.

Figure 14. Staff/Member View – Scholar overview and current activity

The final staff view demonstrates the operational perspective on circulation and complements the student/member workflow.

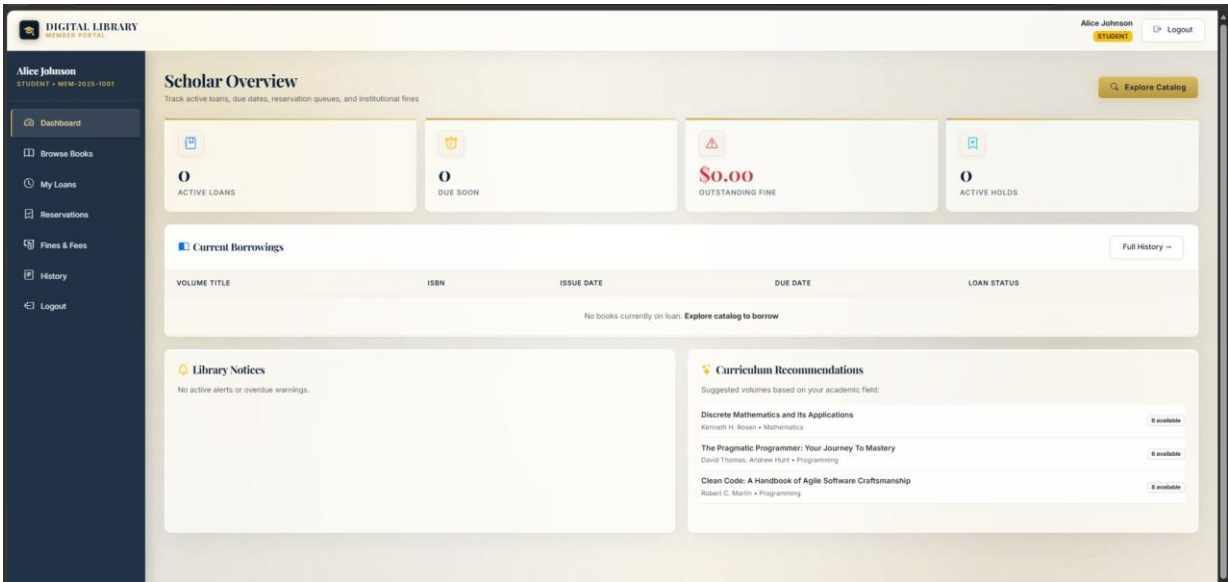


Figure 14. Staff/Member View – Scholar overview and current activity

EVIDENCE INTERPRETATION

The screen demonstrates the corresponding role and workflow described in the report. During evaluation, the evaluator can perform the visible action and then verify the related API/database change. This connects the screenshot evidence to the actual implementation instead of treating the image as standalone decoration.